11. Explain odds and log-odds in logistic regression. What is the cost function used in logistic regression?

Odds and Log-Odds

In Logistic Regression, we want to predict the probability P that an event happens (such as a sample belonging to the positive class). However, predicting probability directly using a straight line is problematic because probabilities are strictly locked between 0 and 1, whereas a straight line extends infinitely. To solve this, we transform probability using two concepts:

- **Odds:** The ratio of the probability that an event *will* happen to the probability that it *will not* happen. For instance, if a horse has an 80% chance of winning a race, its odds of winning are $0.8/0.2 = 4$ (or 4-to-1).

$$\text{Odds} = \frac{P}{1-P}$$

- **Log-Odds (Logit):** The natural logarithm of the odds. While probabilities stop at 0 and 1, and odds stop at 0 and infinity, taking the log curves the values out so that they span from negative infinity to positive infinity.

Logistic regression assumes that this unconstrained **log-odds** relationship is what matches a perfectly straight, linear combination of your input features:

$$\ln\left(\frac{P}{1-P}\right) = \boldsymbol{w}^T\boldsymbol{x} + b$$

The Cost Function: Binary Cross-Entropy (Log Loss)

Because the output of a logistic regression model is a curved probability rather than a straight line, we cannot use Mean Squared Error. Instead, we use a cost function derived from likelihood estimation called **Binary Cross-Entropy** (or Log Loss).

Qualitatively, this function measures the distance between the model's predicted probability distribution and the true 0-or-1 labels. It penalizes a model exponentially if it is confidently wrong—for example, if the model predicts a 99% probability of a sample being positive, but the actual true label is 0, the cost function spikes toward infinity.

12. Why is logistic regression considered a linear classifier?

Logistic Regression is classified as a linear classifier because its internal **decision boundary is a straight line, a flat plane, or a flat hyperplane** in feature space.

Even though the model uses a curved S-shaped sigmoid function to output probabilities between 0 and 1, the classification decision itself relies on a rigid cut-off point. By default, we use a probability threshold of 50% to separate classes:

- If the predicted probability is $\geq 50\%$, predict Class 1.
- If the predicted probability is $< 50\%$, predict Class 0.

If you track this threshold back into the mathematical mechanics of the model, a probability of exactly 50% occurs when the internal linear score equation equals exactly zero:

$$\boldsymbol{w}^T \boldsymbol{x} + b = 0$$

Because this underlying scoring equation is completely linear with respect to your input features, the dividing line where the model switches its prediction from 0 to 1 cannot bend, curve, or loop around data points. It cuts through your feature space as a perfectly flat geometric boundary.

13. Why is Mean Squared Error not preferred for logistic regression?

Using Mean Squared Error (MSE) to optimize a Logistic Regression model causes two major issues:

1. **A Choppy Error Landscape (Non-Convexity):** When you use MSE with standard linear regression, the error landscape forms a clean, single bowl shape where any downhill path leads directly to the absolute lowest error point. However, if you plug the highly curved, non-linear sigmoid activation function into an MSE formula, the resulting error surface warps drastically. It becomes full of false valleys (local minima), plateaus, and deep pockets. If you run gradient descent on this surface, your optimizer can easily get trapped in a bad local minimum, completely failing to find the true best weights.
2. **Severe Optimization Paralysis (Vanishing Gradients):** The sigmoid function is extremely flat at its two outer wings (where the probability approaches exactly 0 or 1). If a model makes a massive mistake—such as predicting a probability near 0 for a sample that is actually a 1—the slope of the sigmoid function at that point is practically zero. In an MSE framework, this flat slope causes the gradient to vanish completely. The algorithm gets paralyzed, incorrectly assuming it does not need to move, and stops learning. Binary Cross-Entropy naturally cancels out this flat-wing effect, maintaining a strong learning signal even during severe mistakes.

14. What is accuracy in classification? Explain precision and recall. What is F1-score? Explain ROC curve and AUC. When is accuracy misleading? Explain false positives and false

negatives. What is class imbalance? Which metric is best for imbalanced datasets?

Confusion Matrix Basics

- **False Positive (Type I Error):** A mistake where the model incorrectly flags a negative sample as positive (e.g., telling a healthy person they are sick).
- **False Negative (Type II Error):** A mistake where the model incorrectly flags a positive sample as negative (e.g., missing a true disease diagnosis).

Standard Metrics

- **Accuracy:** The simple percentage of overall predictions the model got exactly right (both true positives and true negatives combined).
- **Precision:** A measure of reliability. Out of all the samples the model *claimed* were positive, how many were truly positive? (High precision means low false alarms).
- **Recall (Sensitivity):** A measure of coverage. Out of all the actual true positive samples hidden in the dataset, how many did the model successfully find? (High recall means you missed very few cases).
- **F1-Score:** The balanced middle ground. It takes the harmonic mean of precision and recall into a single metric, ensuring a model cannot achieve a perfect score by gaming just one side of the equation.

ROC Curve and AUC

- **ROC Curve:** A graph that visualizes a model's performance across all possible probability cut-off thresholds. It plots the True Positive Rate (Recall) against the False Positive Rate as you shift the threshold from strict to loose.
- **AUC (Area Under Curve):** A single score between 0 and 1 that summarizes the ROC graph. It represents the probability that the model will score a randomly selected positive sample higher than a randomly selected negative one. A perfect classifier has an AUC of 1.0.

Class Imbalance and the Trap of Accuracy

Class Imbalance occurs when your dataset contains a massive amount of one class and very little of another (e.g., a credit card dataset where 99.9% of transactions are legitimate and only 0.1% are fraudulent).

In this scenario, **accuracy is highly misleading**. A broken model that simply guesses "legitimate" for every single transaction achieves a spectacular accuracy score of 99.9%, despite failing to catch a single instance of fraud.

- **Best Metrics for Imbalance:** You should use **F1-Score** or **Precision-Recall curves**. These metrics look closely at the minority class without letting a massive pool of true negatives artificially inflate the model's score.

15. What is the KNN algorithm? Parametric or non-parametric? Explain how KNN classification works. What is the role of K in KNN? Explain distance metrics used in KNN.

Core Concept

The k**-Nearest Neighbors (KNN)** algorithm is an instance-based classifier. It is strictly **non-parametric** because it does not make any assumptions about the underlying distribution of your data, nor does it compress your data into fixed weights or equations. Instead, it memorizes the entire training dataset and uses it as a map to make decisions on the fly.

How it Works

1. A brand-new, unlabeled data point is placed into the feature space.
2. The algorithm measures the physical distance from this new point to every single stored training point.
3. It identifies the K **closest data points** (its nearest neighbors).
4. The neighbors hold a vote. Whichever class has the majority vote among those K neighbors is assigned to the new point.

The Role of K

The hyperparameter K controls the size of your voting neighborhood and dictates the model's complexity:

- **Tiny** K **(e.g.,** $K = 1$**):** High variance and low bias. The decision boundary is highly complex, jagged, and reacts wildly to every single localized outlier or speck of noise (overfitting).
- **Large** K**:** Low variance and high bias. The decision boundary becomes incredibly smooth and broad, which can easily wash out important local nuances (underfitting).

Common Distance Metrics

- **Euclidean Distance:** The standard, straight-line distance between two points measured with a ruler ("as the crow flies").
- **Manhattan Distance:** Grid-like distance traveled along perpendicular axes, similar to walking along city blocks.

16. What information can be obtained from a confusion matrix? Why is accuracy alone sometimes misleading? Explain confusion matrix for multiclass classification. How does a confusion matrix help evaluate model performance?

Qualitative Breakdown

A **confusion matrix** is a structured scoreboard that tells you exactly how and where your classifier is making mistakes. Instead of giving you a single broad percentage score, it categorizes every prediction into a grid of actual vs. predicted classes. For a binary problem, it shows your exact counts of True Positives, True Negatives, False Positives, and False Negatives.

As established in Question 14, relying solely on an accuracy percentage can mask severe underlying flaws if your dataset has a class imbalance. The confusion matrix reveals this blind spot by showing if your model is achieving a high score simply by completely ignoring the minority class.

Multiclass Confusion Matrix

In a multiclass problem (e.g., sorting images into Cats, Dogs, or Foxes), the matrix scales up into an $N \times N$ grid:

- The **main diagonal line** running from the top-left to the bottom-right corner displays your correct predictions—where the actual label matches the predicted label.
- Any numbers sitting outside this diagonal line represent specific classification errors. For instance, looking at the intersection of row `Cat` and column `Dog` tells you exactly how many times the model confused a cat for a dog. This granular visibility helps you adjust feature engineering for specific classes that look too similar to the model.

17. What are support vectors? What is the margin in SVM? What is the difference between hard margin and soft margin SVM? Why do we maximize the margin in SVM? What is the role of the regularization parameter C?

Key Definitions

- **Support Vectors:** The critical, outermost data points of each class that sit closest to the dividing boundary. They act as the structural pillars of the model; if you remove them, the entire dividing hyperplane shifts. Any data points sitting further back are ignored.
- **The Margin:** The clear buffer zone or "no-man's-land" distance between the dividing hyperplane and the closest support vectors.

Hard Margin vs. Soft Margin

- **Hard Margin SVM:** Assumes the data can be perfectly separated by a clean straight line. It strictly forbids any mistakes or boundary crossings during training. It fails completely if any classes overlap or contain noise.
- **Soft Margin SVM:** Introduces flexible **slack variables**. This allows a controlled amount of error, permitting noisy outliers to cross into the margin or be misclassified if it helps build a cleaner overall boundary line.

Why Maximize the Margin?

Maximizing the margin reduces structural risk. A wider margin leaves the largest possible safety clearing between classes, making the model far less sensitive to minor data variations and giving it a much higher chance of correctly classifying new data.

The Role of Parameter C

The hyperparameter C acts as a penalty dial for margin violations:

- **Large C:** Severely penalizes any mistakes. The model prioritizes avoiding classification errors over a wide margin, creating a narrow boundary that fits the training data tightly but can overfit easily (high variance).
- **Small C:** Highly permissive of margin violations. The model prioritizes a wide, simple margin, accepting a few misclassifications to build a more robust, stable decision boundary (high bias).

18. What is the kernel trick in SVM? Why do we use kernels in SVM? Explain linear kernel vs nonlinear kernel. What are common kernel functions? What is the RBF (Gaussian) kernel? How does kernel SVM handle nonlinear data?

The Kernel Trick and Nonlinear Data

Imagine a circular target of red dots surrounded by a ring of blue dots on a flat sheet of paper. You cannot draw a straight line to separate them.

Instead of manually calculating complex high-dimensional mathematical coordinates explicitly, Support Vector Machines use the **Kernel Trick**. A kernel function mathematically calculates the similarity between points in a simulated, higher-dimensional space *implicitly*, without ever actually transforming the data coordinates.

In this higher-dimensional space, the data becomes cleanly, linearly separable by a flat hyperplane. When that flat boundary is projected back down to your original lower-dimensional

space, it manifests as a curved, non-linear boundary.

Linear vs. Nonlinear Kernels

- **Linear Kernel:** Computes a simple baseline dot product in your original space. Used when data is already linearly separable.
- **Nonlinear Kernel:** Maps relationships into richer dimensional configurations to capture complex, curving classification boundaries.

Common Kernels and the RBF Kernel

Common options include Polynomial and **Radial Basis Function (RBF)** kernels.

The RBF (Gaussian) kernel handles complex spaces by mapping data into an **infinite-dimensional space**. It acts as a localized proximity sensor: if two points are close to each other in the feature space, the kernel outputs a high similarity score; if they are far apart, the score drops to zero. This allows the SVM to draw tight, custom-shaped boundary pockets around clusters of data.

19. Write the optimization objective of SVM. What are slack variables in soft margin SVM? What is the Lagrangian formulation of SVM? Explain the constraint conditions in SVM.

The Optimization Objective and Slack Variables

The primary goal of a soft-margin SVM is to maximize the safety margin while minimizing classification errors. This is expressed as a minimization problem:

$$\min_{\boldsymbol{w},b} \frac{||\boldsymbol{w}||^2}{2} + C\sum_i \xi_i$$

Subject to the structural constraint that all points sit on the correct side of the margin : $y_i(\boldsymbol{w}\boldsymbol{x}_i + b) \geq 1 - \xi_i$.

The **slack variables (ξ_i)** quantify exactly how much a data point violates the ideal margin boundary:

- $\xi_i = 0$: The point sits safely on or outside the proper margin line.
- $0 < \xi_i \leq 1$: The point crossed into the margin buffer zone but is still on the correct side of the dividing line.
- $\xi_i > 1$: The point crossed the line entirely and is misclassified.

Lagrangian Formulation and Constraints

To solve this constrained optimization problem, we use a mathematical technique called the **Lagrangian formulation**. It blends the main objective function and its constraints into a single unconstrained equation using multipliers (α_i).

This allows us to convert the complex problem into its **Dual Form**, where the optimization loop no longer looks at the features directly, but instead focuses entirely on the dot-product similarity between support vectors.

The solution must satisfy the **Karush-Kuhn-Tucker (KKT) conditions**:

- If a point sits safely outside the margin, its multiplier α_i drops to exactly 0, meaning it has zero influence on the model.
- If $0 < \alpha_i < C$, the point is a support vector lying exactly on the margin edge.
- If $\alpha_i = C$, the point is a support vector that has violated the margin boundary.

20. What happens when data is not linearly separable? How does SVM handle outliers?

When Data is Not Linearly Separable

If a dataset cannot be cleanly divided by a straight line or a flat plane, a strict hard-margin SVM will fail completely to find a solution. SVM handles non-linear datasets using a two-pronged approach:

1. **The Soft Margin Framework:** It uses slack variables to allow a reasonable, controlled number of training mistakes rather than breaking down.
2. **The Kernel Trick:** It applies non-linear kernels (like the RBF kernel) to project the data into a higher-dimensional space where a clean, linear separation can be achieved.

Handling Outliers

SVM handles anomalous outliers through its soft-margin optimization adjustments via the regularizing hyperparameter C:

- An outlier lying far inside the opposing class territory will demand a massive slack penalty value ($\xi_i > 1$).
- If C is configured to be **small**, the optimization loop prioritizes maximizing the margin width over correcting single anomalies. The model will simply ignore the outlier, allowing it to be misclassified to maintain a stable, well-generalized decision boundary.

- If C is configured to be **large**, the model penalizes that outlier's violation aggressively. This forces the decision boundary to bend or narrow significantly just to accommodate the outlier, which often leads to overfitting.